

Sequence Labelling & Neural Language Models

HMMs, POS Tagging, Viterbi | Feedforward
Networks, RNNs, and GRUs



- Time: ~1 min
- Say:
- Good evening everyone. Today we move from simple language models into sequence labelling and neural language models. The first half connects to POS tagging, NER, HMMs, and Viterbi. The second half moves into neural models: feedforward networks, RNNs, GRUs, and finally text generation. My goal is not to make you memorise formulas. My goal is that you can read the code, understand what each block is doing, and connect it to the assignment/lab style questions.

Agenda

- Sequence labelling as a problem class; POS tagging, NER, BIO scheme
- HMMs and the Viterbi algorithm — read through a working implementation
- Feedforward neural LM (Bengio 2003) and gradient descent
- RNNs and the vanishing gradient problem
- GRUs and bidirectional GRUs — the TMA Q4 architecture
- Generating text: greedy decoding, beam search, and temperature

- Time: ~2 min
- Say:
- This is the roadmap for today. We start with sequence labelling: instead of one label for the whole sentence, we predict one label per word. Then we look at HMMs and Viterbi, mainly to understand the dynamic programming idea. After that, we revisit feedforward neural language models and the basic training loop. Then we discuss why RNNs were introduced, why vanilla RNNs struggle with long sequences, and how GRUs help. Finally, we look at decoding: greedy search, beam search, and temperature. Keep this big picture in mind: we are moving from counting-based models to neural models that learn representations.

Segment 1 — Sequence Labelling as a Problem Class

- Text classification: one label per document
- Sequence labelling: one label per token in the sequence
- Canonical tasks:
 - POS tagging: "The dog runs fast." → DET NOUN VERB ADV
 - Named entity recognition (NER): "Barack Obama was born in Hawaii." → PER PER O O O LOC
 - Chunking: grouping tokens into phrases (NP, VP, ...)

- Time: ~2.5 min
- Say:
- Let us first define sequence labelling. In text classification, the whole document or sentence gets one label. For example, an SMS is ham or spam. In sequence labelling, every token gets its own label. POS tagging is a simple example: 'The dog runs fast' becomes determiner, noun, verb, adverb. NER is another example: 'Barack Obama' is a person and 'Hawaii' is a location. Chunking groups words into phrases. The key difference is that the output length is linked to the input length: one prediction per token.

BIO Tagging Scheme for NER

- B-TAG: Beginning of an entity of type TAG
- I-TAG: Inside (continuation of) an entity of type TAG
- O: Outside — not part of any entity
- Example: "Barack Obama was born in Hawaii"
 - B-PER I-PER O O O B-LOC
- Why not just PER, PER, O, O, O, LOC?
 - "Barack Obama Michelle Obama" would be PER PER PER PER — two people or one? BIO makes boundaries explicit.

- Time: ~3 min
- Say:
- For named entity recognition, we often use the BIO tagging scheme. B means beginning of an entity, I means inside an entity, and O means outside any entity. In 'Barack Obama was born in Hawaii', Barack is B-PER, Obama is I-PER, and Hawaii is B-LOC. The reason we need B and I is boundary clarity. Suppose we have 'Barack Obama Michelle Obama'. If we only use PER PER PER PER, we do not know whether it is one long name or two different names. BIO makes the start and continuation explicit.

Exercise

- Tag this sentence using the BIO scheme:
- "Apple CEO Tim Cook announced the new iPhone at the Yerba Buena Center."
- Expected answer:
 - B-ORG O B-PER I-PER O O O B-PROD I-PROD O O B-LOC I-LOC I-LOC
- Common mistakes: missing that "Tim Cook" is two tokens (B-PER I-PER), or using PER for both without the B/I distinction

- Time: ~2.5 min
- Say:
- Let us do a quick exercise. The sentence is: 'Apple CEO Tim Cook announced the new iPhone at the Yerba Buena Center.' Apple is an organisation, CEO is not an entity here, Tim Cook is a person with two tokens, iPhone is a product, and Yerba Buena Center is a location with three tokens. So the expected BIO labels are B-ORG, O, B-PER, I-PER, O, O, O, B-PROD, I-PROD, O, O, B-LOC, I-LOC, I-LOC. The common mistake is to write PER PER without marking the first token and continuation.

Segment 2 — HMMs, Just Enough to Understand Viterbi (optional)

- States (hidden): POS tags — DET, NOUN, VERB, ADJ, ...
- Observations (visible): the actual words
- Transition probabilities A: $P(\text{tag}_t | \text{tag}_{t-1})$ — e.g. "DET NOUN" common, "NOUN NOUN" rare
- Emission probabilities B: $P(\text{word} | \text{tag})$ — e.g. $P(\text{"runs"} | \text{VERB})$ high, $P(\text{"runs"} | \text{DET})$ near zero
- Why not just pick the best tag greedily, word by word?
 - A locally-good choice can turn out globally wrong once you see the next word
- The one limitation to remember: first-order Markov assumption
 - Transition table only looks at the immediately preceding tag — cannot capture non-local dependencies (e.g. subject-verb agreement across a clause)



- Time: ~3 min
- Say:
- Now we move to HMMs, only as much as needed for Viterbi. In an HMM, the hidden states are the tags, such as DET, NOUN, VERB, and so on. The observations are the actual words. We need transition probabilities: how likely is one tag after another tag? For example, DET followed by NOUN is common. We also need emission probabilities: how likely is a word given a tag? For example, 'runs' as a verb is more likely than 'runs' as a determiner. The important point is: choosing the best tag word by word may fail because the best local choice may not give the best full sentence. One limitation is the first-order Markov assumption: it only looks at the immediately previous tag.

Three Probability Tables (optional)

- Initial state probabilities π : $P(\text{tag at position } 1)$
 - "The" is almost always DET, so $\pi(\text{DET})$ is high
- Transition probabilities A: $P(\text{tag}_t \mid \text{tag}_{t-1})$
 - "DET NOUN" is common; "NOUN NOUN" is rare — learned by counting transitions in a tagged corpus
- Emission probabilities B: $P(\text{word} \mid \text{tag})$
 - $P(\text{"runs"} \mid \text{VERB})$ is high; $P(\text{"runs"} \mid \text{DET})$ is near zero

- Time: ~2 min
- Say:
- An HMM has three probability tables. The first is the initial probability, π , which says which tag is likely at the first position. The second is the transition table A, which says how likely one tag is after another. The third is the emission table B, which says how likely a word is under a tag. In practice, these can be learned by counting from a tagged corpus. For this lecture, do not worry about estimating them perfectly. Just remember: initial tells us how to start, transition tells us how tags move, and emission tells us how tags generate words.

Segment 3 — Viterbi: Reading the Code (optional)

- The idea in one sentence: build a trellis (best score so far, per tag, per position) plus a backpointer table, then walk forward and backtrack
- Viterbi vs. the forward algorithm:
 - Viterbi takes the max over previous-tag paths — finds the single best tag sequence
 - Forward algorithm takes the sum — finds the total probability of the sentence
- Same table structure, different operator — this is the classic quiz confusion

- Time: ~2.5 min
- Say:
- Viterbi is the algorithm that finds the best hidden tag sequence. The idea is simple: build a table, often called a trellis. For each word position and each possible tag, store the best score so far. Also store a backpointer: which previous tag gave that best score. We move forward through the sentence, filling the table. At the end, we pick the best final tag and backtrack. A common confusion is Viterbi versus the forward algorithm. Viterbi takes a max and finds one best path. Forward takes a sum and finds total probability.

```

def viterbi(observations, tags, pi,
            transitions, emissions):
    T = len(observations)
    viterbi_table = [{} for _ in range(T)]
    backpointer = [{} for _ in range(T)]

    # Step 1 - initialise at t=0
    word = observations[0]
    for tag in tags:
        emit = emissions[tag].get(word, 1e-10)
        viterbi_table[0][tag] = (
            math.log(pi[tag]) + math.log(emit))
        backpointer[0][tag] = None

    # Step 2 - recurrence
    for t in range(1, T):
        word = observations[t]
        for tag in tags:
            emit = emissions[tag].get(word, 1e-10)
            best_prev_tag, best_score = None, float("-inf")
            for prev_tag in tags:
                trans = transitions[prev_tag].get(tag, 1e-10)
                score = (viterbi_table[t-1][prev_tag]
                        + math.log(trans))
                if score > best_score:
                    best_prev_tag, best_score = prev_tag, score
            viterbi_table[t][tag] = best_score + math.log(emit)
            backpointer[t][tag] = best_prev_tag

```



Viterbi in Code (1/2) — Setup and Recurrence

- Time: ~3.5 min
- Say:
- Let us read this code slowly. First, T is the number of observations. Then we create two lists of dictionaries: `viterbi_table` and `backpointer`. At time zero, we initialise every possible tag. The score is log initial probability plus log emission probability for the first word. Then comes the recurrence. For each later word and each current tag, we check all possible previous tags. We compute the previous best score plus the transition score. We keep the previous tag that gives the maximum score. After choosing the best previous tag, we add the emission score for the current word. This fills one cell of the trellis.

```

# Step 3 - best final tag
best_final_tag = max(
    tags, key=lambda tag: viterbi_table[T-1][tag])

# Step 4 - backtrack through backpointers
best_path = [best_final_tag]
current_tag = best_final_tag
for t in range(T - 1, 0, -1):
    current_tag = backpointer[t][current_tag]
    best_path.insert(0, current_tag)

return best_path

## Run it once and show the output:
sentence1 = ["the", "dog", "runs"]
path = viterbi(sentence1, tags, pi,
               transitions, emissions)
print(f"Best path: {path}")

```

Viterbi in Code (2/2) — Best Tag and Backtrack

- Time: ~2 min
- Say:
- This second part completes the algorithm. Once the table is filled, we find the best final tag by looking at the last position. But the final tag alone is not enough; we need the whole path. So we start from the best final tag and repeatedly follow the backpointer backwards. Each backpointer says: to reach this tag at this position, which previous tag was best? Finally, we insert each recovered tag at the front of the path. The output is the best tag sequence for the whole sentence, for example the best path for 'the dog runs'.

What to Notice in the Code (optional)

- The recurrence:
 - `score = viterbi_table[t-1][prev_tag] + log(trans)`, then separately `+ log(emit)`
 - The emission term is added after the max over previous tags, not inside the loop — it doesn't depend on `prev_tag`
- Why we need backpointer, not just `viterbi_table`
 - `viterbi_table` alone gives the score of the best path, not the tags that produced it
- Where the trellis and backpointer "are" in the code
 - Just lists of dicts, one entry per time step — the abstract grid from the textbook is this concrete data structure

- Time: ~2 min
- Say:
- There are three important things to notice in the code. First, the transition score depends on the previous tag, so it is inside the loop over previous tags. The emission score depends only on the current tag and current word, so it can be added after the max. Second, the viterbi table only stores scores. It does not remember the actual path. That is why we need the backpointer. Third, the textbook trellis is not magic. In code, it is just a list of dictionaries: one dictionary per time step.

The One Limitation, and Where Viterbi Leads (optional)

- First-order Markov assumption: only the immediately preceding tag matters
 - Cannot capture non-local dependencies (e.g. subject-verb agreement across a clause)
- Where this idea reappears: beam search (later this lecture)
 - Same "extend partial solutions, score, keep the best, backtrack" structure
 - Except beam search can't afford to try every previous option — we'll see why

- Time: ~1.5 min
- Say:
- The main limitation is the first-order Markov assumption. The current tag depends only on the immediately previous tag. This is simple and efficient, but it cannot model long-range dependency well. The useful idea that survives is the search pattern: extend partial solutions, score them, keep the best, and then recover the answer. We will see a related idea again when we discuss beam search later. Viterbi can search exactly in a small structured space; beam search is an approximate version used when the space is much larger.

Segment 4 — Why N-gram LMs Cannot Generalise

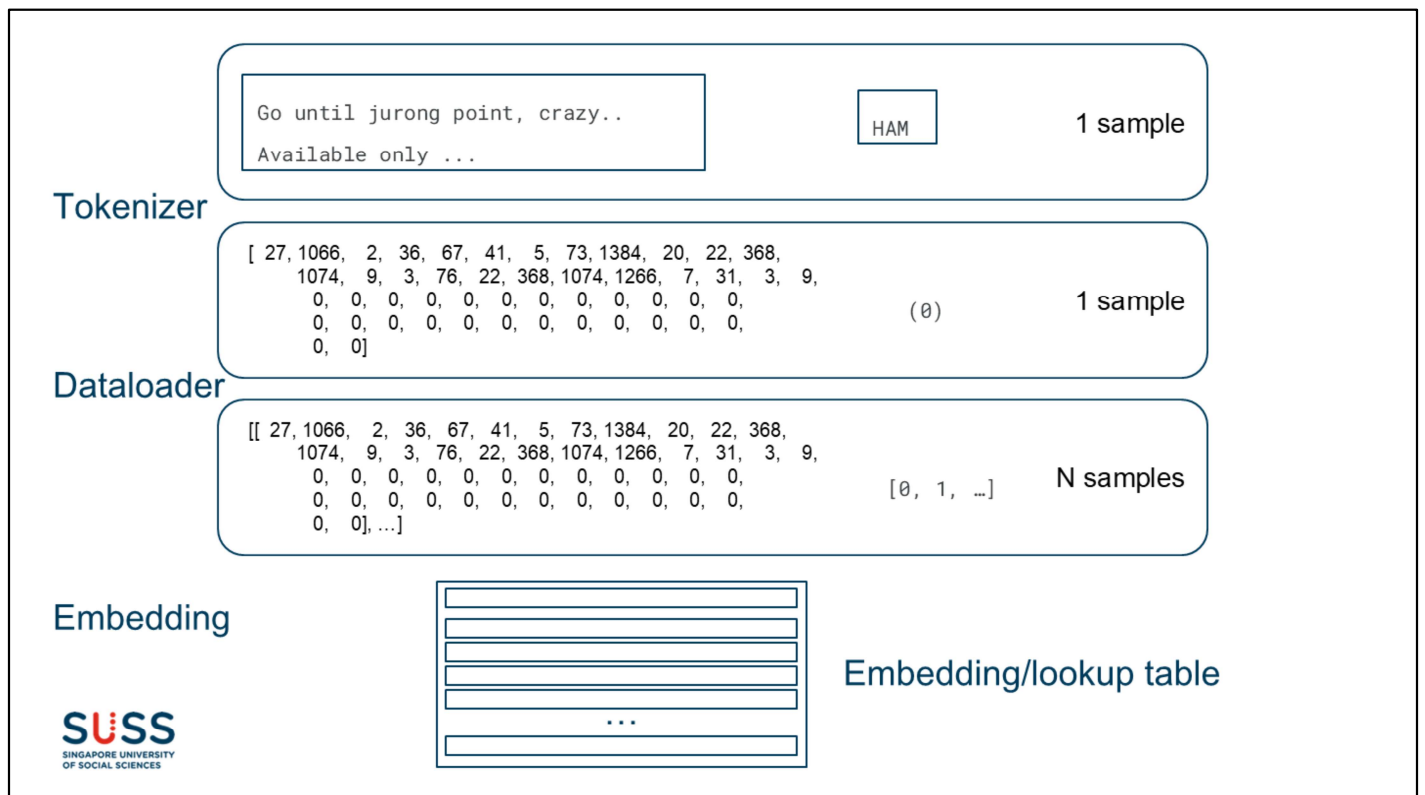
- "the cat sat" and "the dog sat" are completely independent to a bigram LM
- If "dog sat" is unseen, $P(\text{sat} \mid \text{dog})$ falls back to smoothing — same as $P(\text{sat} \mid \text{cat})$
- The model cannot generalise that "cat" and "dog" behave similarly as subjects

- Time: ~2.5 min
- Say:
- Now we move from HMMs and tagging to language models. The weakness of n-gram models is that they do not generalise well. To a bigram model, 'the cat sat' and 'the dog sat' are separate events. If 'dog sat' was never seen, the model cannot naturally use the fact that dog and cat are both nouns and behave similarly. It falls back to smoothing. So n-gram models are good at memorising local counts, but weak at sharing knowledge across similar words.

Bengio et al. (2003) — The First Neural Language Model

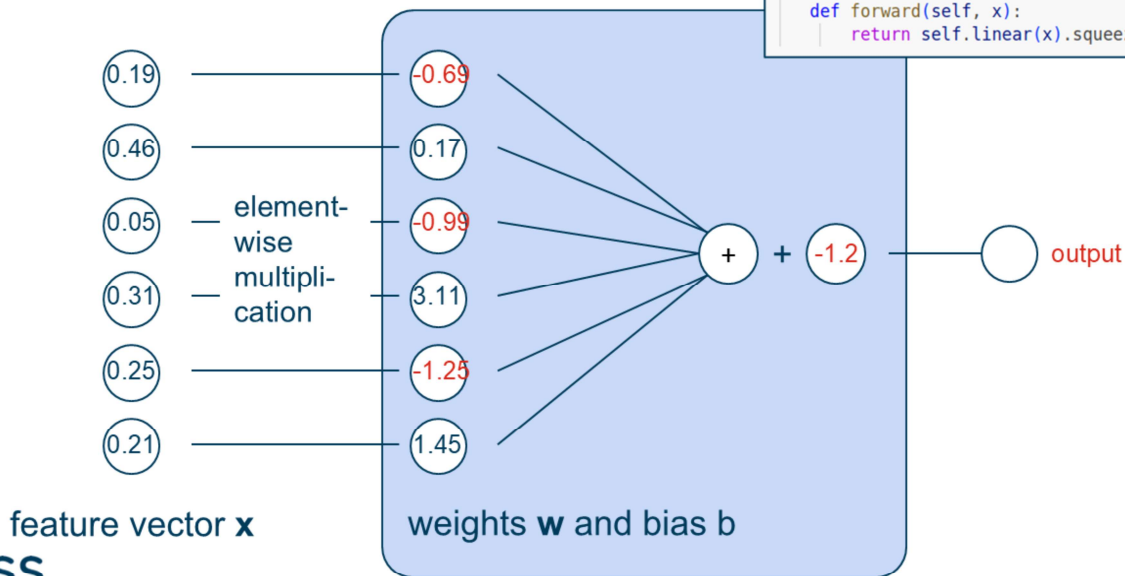
- Key idea: represent each word as a dense vector (embedding)
 - Words with similar embeddings produce similar predictions — "cat" and "dog" → similar contexts predicted
- Architecture:
 - 1. Look up the embedding for each context word $w_{i-k} \dots w_{i-1}$
 - 2. Concatenate the context embeddings into one vector
 - 3. Pass through a hidden layer with a non-linearity (tanh or ReLU)
 - 4. Linear layer + softmax over the vocabulary to predict the next word
- Still a fixed-context (Markov) model — generalisation comes from the shared embedding space

- Time: ~3.5 min
- Say:
- Bengio's neural language model introduced a key idea: represent each word as a dense vector, called an embedding. Similar words can have similar embeddings, so the model can generalise. The architecture is: look up embeddings for the context words, concatenate them, pass them through a hidden layer, and then use a softmax layer to predict the next word. This is still a fixed-context model because it only uses the last k words. But the important improvement is that the model learns a shared representation space instead of treating every word combination as completely independent.



- Time: ~2 min
- Say:
- This diagram connects raw text to embeddings. A sentence starts as text. The tokenizer converts each word into an integer ID. Padding may be added so that examples have the same length. The dataloader groups many examples into batches. Then the embedding layer works like a lookup table: each integer ID selects one row from the embedding matrix. That row is the vector representation of the word. So embeddings are not manually written features; they are trainable rows in a matrix.

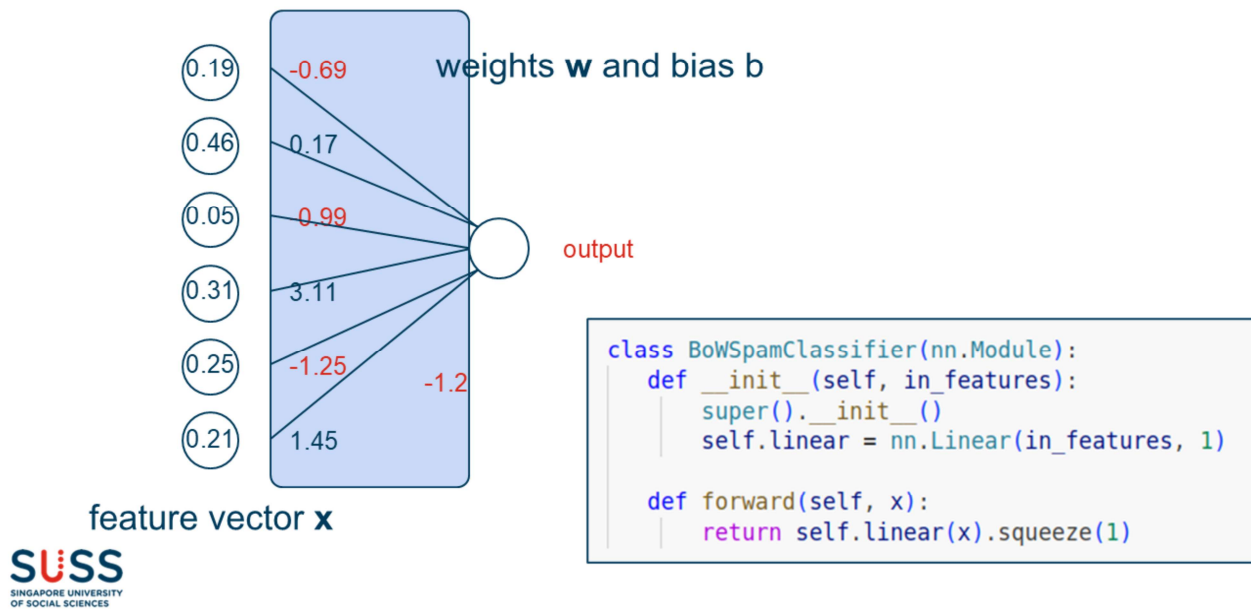
Re-cap the Linear model



16

- Time: ~2 min
- Say:
- Before deeper networks, let us recap the linear model. We have a feature vector x . Each feature is multiplied by a corresponding weight. Then all products are summed and a bias is added. The result is the model output, also called a logit before applying sigmoid or softmax. In PyTorch, `nn.Linear` does exactly this operation. For binary spam classification, the output can be a single logit. Positive or negative values are later converted into a probability using sigmoid.

A simplified drawing

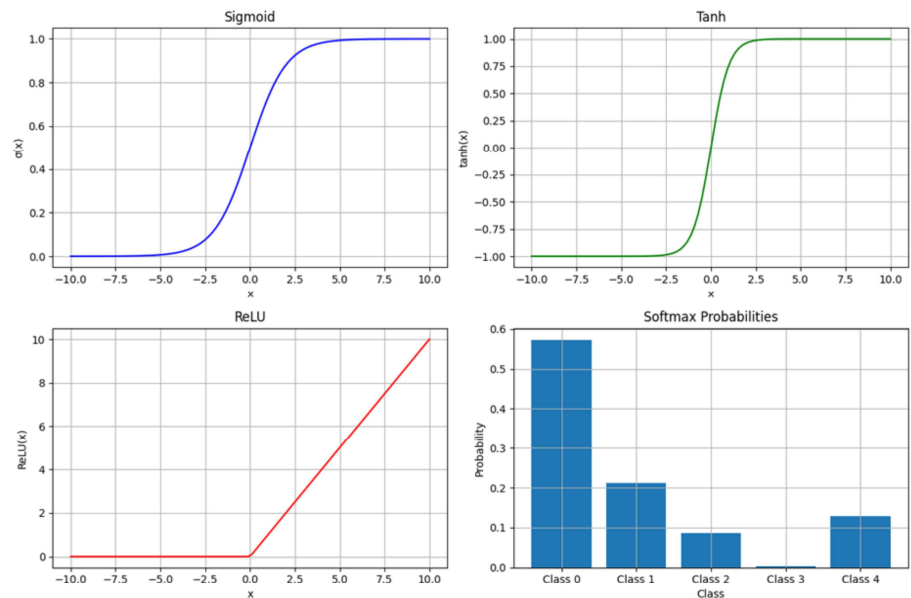


17

- Time: ~1 min
- Say:
- This is the same linear model drawn in a cleaner way. The circles on the left are input features. The box contains the weights and bias. The output is one number. This simplified picture is useful because neural networks are built by repeating and stacking this idea. A layer is basically many weighted sums, followed by a non-linearity.

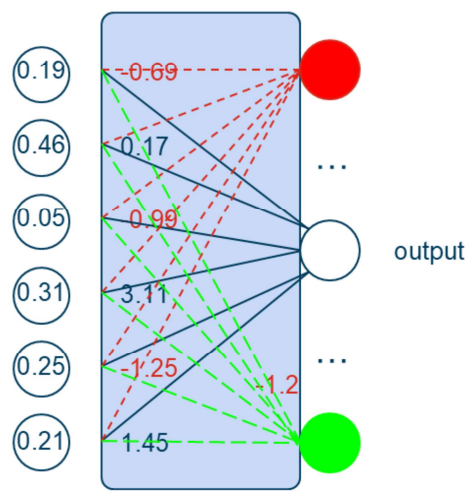
Loss function

- The loss function encourage the weights to “separate” the different classes by a greater margin.



- Time: ~1.5 min
- Say:
- The loss function tells the model how wrong it is. During training, we change the weights to reduce the loss. For classification, the loss should encourage the correct class to get a higher score and the wrong classes to get lower scores. The plots show common functions like sigmoid, tanh, ReLU, and softmax. Softmax is important when we have multiple classes because it converts raw scores into a probability distribution.

What if it's 10-classification?



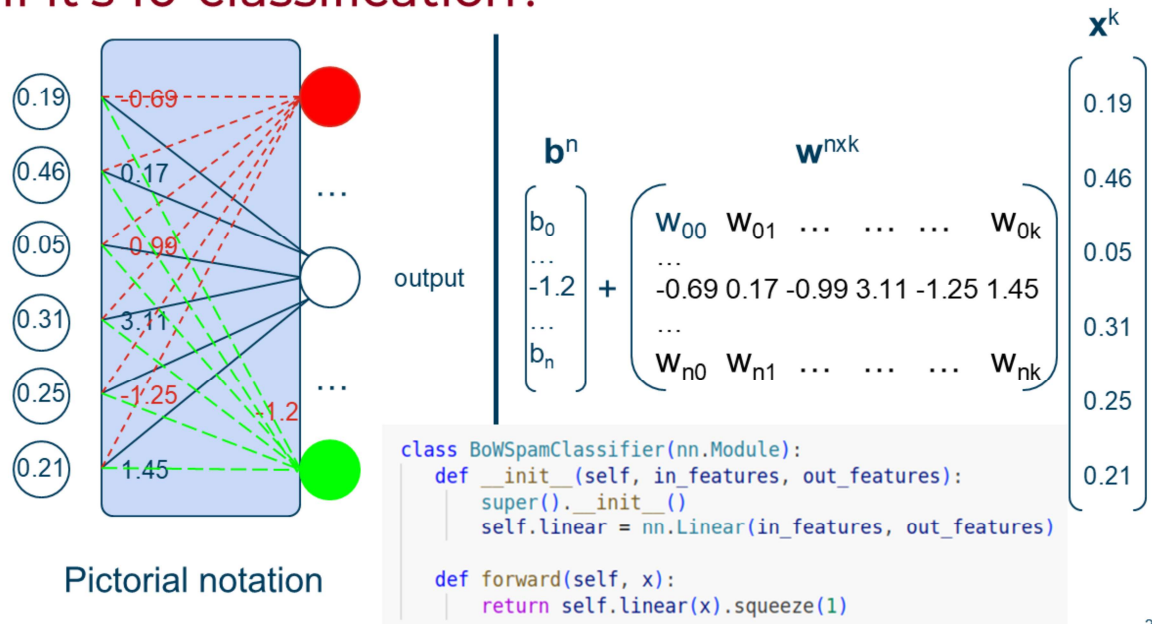
Pictorial notation

$$\begin{bmatrix} b_0 \\ \dots \\ -1.2 \\ \dots \\ b_n \end{bmatrix} + \begin{bmatrix} w_{00} & w_{01} & \dots & \dots & \dots & w_{0k} \\ \dots & \dots & \dots & \dots & \dots & \dots \\ -0.69 & 0.17 & -0.99 & 3.11 & -1.25 & 1.45 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ w_{n0} & w_{n1} & \dots & \dots & \dots & w_{nk} \end{bmatrix} \begin{bmatrix} x^k \\ 0.19 \\ 0.46 \\ 0.05 \\ 0.31 \\ 0.25 \\ 0.21 \end{bmatrix}$$

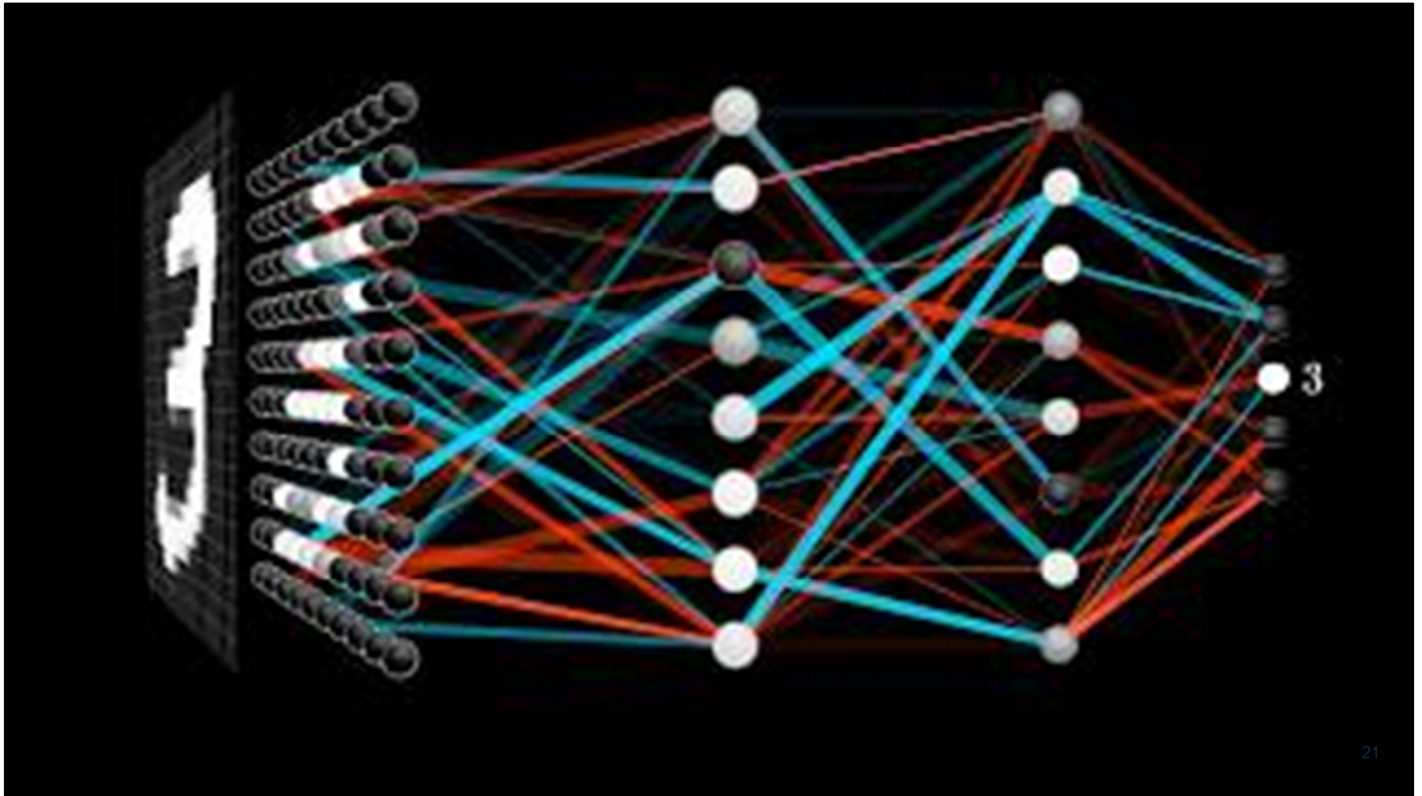
Mathematical notation

- Time: ~2 min
- Say:
- If we have ten classes instead of two, the model needs ten outputs. Each output corresponds to one class. Mathematically, this means the weight is no longer just one vector. It becomes a matrix. The input vector is multiplied by the weight matrix, and a bias vector is added. The result is a vector of class scores. These scores are not probabilities yet. They are logits, and we later apply softmax or use a loss function that internally handles softmax.

What if it's 10-classification?

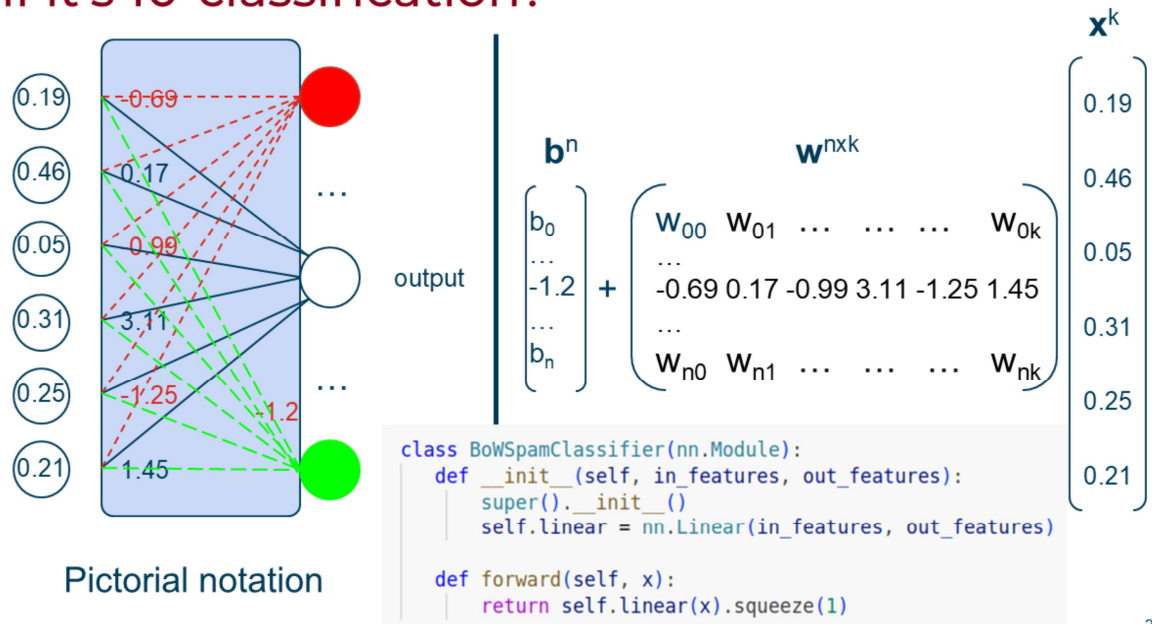


- Time: ~1 min
- Say:
- This slide adds the PyTorch view. In code, we only change `nn.Linear` from output size 1 to output size equal to the number of classes. So for ten classes, `out_features` becomes 10. This is a useful mental model: the same layer can support binary classification, multi-class classification, or language modelling, depending on how many outputs we ask for.



- Time: ~0.5 min
- Say:
- This image is just a visual bridge from a single linear classifier to a deeper neural network. Instead of connecting inputs directly to outputs, we can insert hidden layers. Hidden layers allow the model to learn intermediate representations, not just one direct weighted sum.

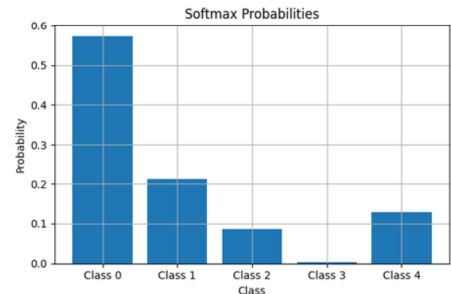
What if it's 10-classification?



- Time: ~1 min
- Say:
- We return to the ten-class picture. The important point is that multi-class classification produces one score per class. The model does not directly output the class name. It outputs a vector of scores, and the highest score is usually selected as the prediction after training.

What should be the loss function for 10-classification?

- ❑ When the model is randomly initialized, what is the range of the output values?
- ❑ The desired output values should be the probability of each class
- ❑ Softmax function ensures a probability distribution



- Time: ~2 min
- Say:
- Now the question is: what loss should we use for ten-class classification? When the model is randomly initialised, its raw outputs can be any real values. But we want something that behaves like probabilities across classes. Softmax converts the logits into values between zero and one that sum to one. For training, we compare this predicted distribution with the true class. If the true class is class 3, then ideally class 3 should get probability close to one and all others close to zero.

Cross entropy loss

- How to properly compare the difference between two distribution?

$$H(p, q) = - \sum_{x \in \mathcal{X}} p(x) \log q(x).$$

- Time: ~2 min
- Say:
- Cross entropy is the standard loss for comparing a true distribution with a predicted distribution. In classification, the true distribution is usually one-hot: probability one for the correct class and zero for others. The predicted distribution comes from softmax. Cross entropy becomes small when the model gives high probability to the correct class. It becomes large when the model gives low probability to the correct class. In PyTorch, CrossEntropyLoss combines log-softmax and negative log-likelihood, so we pass raw logits, not probabilities.

```

import torch.nn as nn

class BoWSpamClassifier(nn.Module):
    def __init__(self, in_features):
        super().__init__()
        self.linear = nn.Linear(in_features, 1)

    def forward(self, x):
        return self.linear(x).squeeze(1)  # logits (no sigmoid here)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = BoWSpamClassifier(vocab_size)
model.to(device)

```

- Time: ~1.5 min
- Say:
- This is the feedforward spam classifier in PyTorch. It inherits from `nn.Module`. In the constructor, we define one linear layer. In the forward method, we pass `x` through the linear layer and squeeze the output shape. Notice the comment: logits, no sigmoid here. This is important because `BCEWithLogitsLoss` expects raw logits and applies the stable sigmoid operation internally. The model is then moved to GPU if available, otherwise CPU.

```
pos_weight = torch.tensor([ (len(y_train) - y_train.sum()) / y_train.sum() ])
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight) # >1 ⇒ penalise spam
errors more
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

- Time: ~1.5 min
- Say:
- This slide defines the loss and optimizer. `pos_weight` handles class imbalance. In spam detection, spam messages may be fewer than ham messages. A positive weight greater than one makes mistakes on spam more costly. `BCEWithLogitsLoss` is suitable for binary classification with raw logits. The optimizer is Adam with learning rate 0.001. Adam is a common default because it is usually more stable than plain SGD for many beginner experiments.

```
def run_epoch(loader, training=False):
    total_loss = 0
    correct = 0
    total = 0

    for x, y in loader:
        x = x.to(device)
        y = y.to(device)
        logits = model(x)
        loss = criterion(logits, y)
        if training:
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
        total_loss += loss.item() * len(y)
        preds = (torch.sigmoid(logits) > .5).long()
        correct += (preds == y.long()).sum().item()
        total += len(y)

    return total_loss/total, correct/total
```

- Time: ~2.5 min
- Say:
- This is one training or evaluation epoch. For each batch, we move x and y to the device, compute logits, and compute the loss. If training is true, we clear old gradients, backpropagate using `loss.backward`, and update the parameters using `optimizer.step`. Then we accumulate the loss and accuracy. For prediction, we apply sigmoid to convert logits into probabilities and use 0.5 as the threshold. This function is useful because the same code can run training and validation by changing one flag.

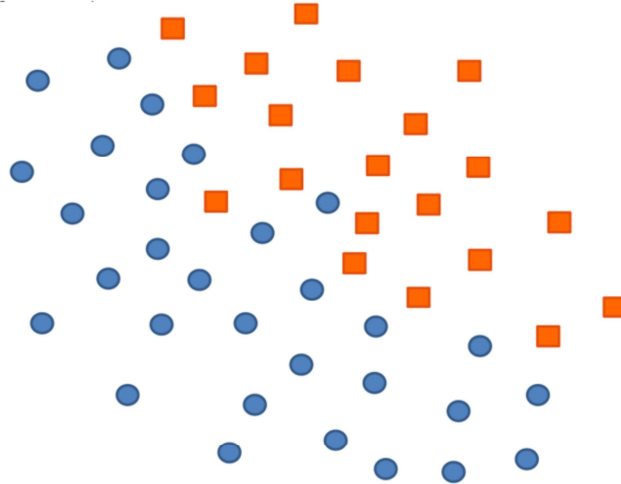
```
for epoch in range(10):
    train_loss, train_acc = run_epoch(train_loader, training=True)
    val_loss, val_acc = run_epoch(val_loader)
    print(f'E{epoch:02d} train_loss={train_loss:.4f} val_acc={val_acc:.4f}')

print("Final test accuracy:", run_epoch(test_loader))
```

- Time: ~1 min
- Say:
- Here we repeat the epoch several times. Each epoch trains once over the training data, then evaluates on validation data. The printed values help us see whether loss is decreasing and validation accuracy is improving. After training, we test once on the test set. The key principle is: train many times, tune using validation, and report final performance using test.

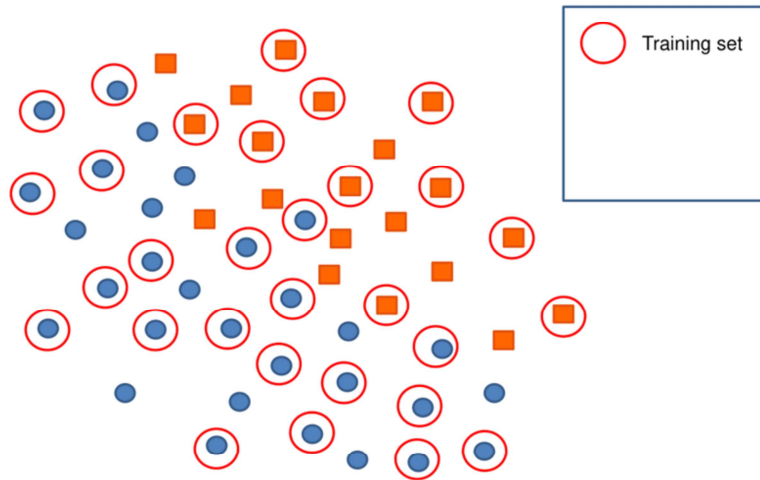
Training, Validation and Test

Suppose these data points are all we have, and we want to use them to access the algorithm's performance on new unseen data



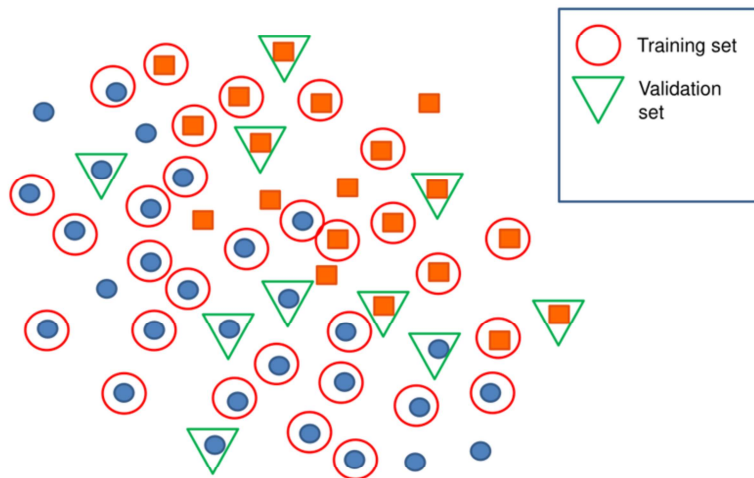
- Time: ~1.5 min
- Say:
- Now let us review training, validation, and test. Suppose these points are all the data we have. We want to estimate how well the model performs on new unseen data. If we train and test on the same examples, the result will be too optimistic. The model may simply memorise. So we split the data to simulate future unseen cases.

Training, Validation and Test



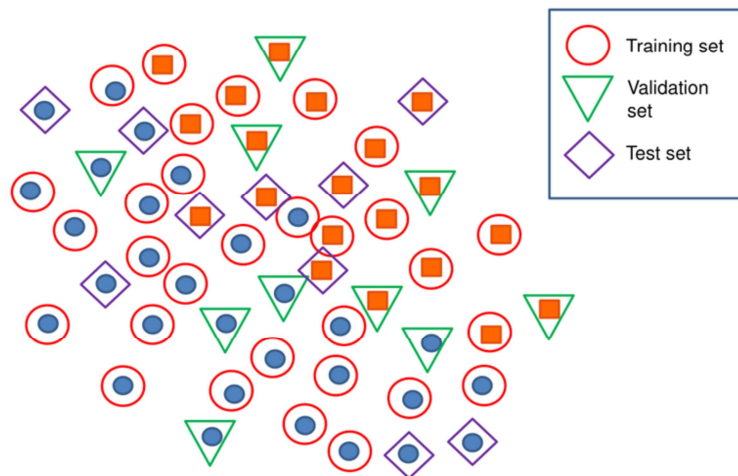
- Time: ~0.75 min
- Say:
- The red circles show the training set. These are the examples used to update the model weights. The model sees these examples directly during training. Because of that, training performance alone is not enough to judge generalisation.

Training, Validation and Test



- Time: ~0.75 min
- Say:
- The green triangles represent the validation set. The validation set is not used to update weights directly. We use it to choose hyperparameters, such as learning rate, number of epochs, or model size. It acts like a practice exam.

Training, Validation and Test



- Time: ~0.75 min
- Say:
- The purple diamonds represent the test set. The test set should be touched only at the end. It is used for the final estimate of performance. If we keep using the test set to tune the model, it stops being a true test set.

Training, Validation and Test



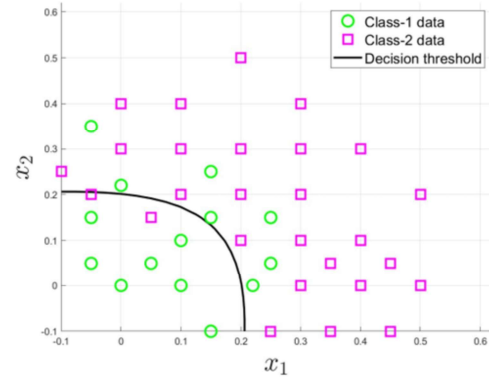
N-fold Cross-validation

- When $N=m$ (training sample size), then it is called a leave-one-out validation.

- Time: ~1.5 min
- Say:
- Cross-validation is useful when the dataset is small. Instead of using only one train-validation split, we rotate which part is used for validation. In four-fold cross-validation, we train four times, each time using a different fold for validation. If the number of folds equals the number of training samples, it is called leave-one-out validation. It gives more stable estimates, but it is more computationally expensive.

Evaluation Metrics - Classification

	Class-1 (predicted)	Class-2 (predicted)
Class-1 (actual)	7 (TP)	7 (FN) Type-II error
Class-2 (actual)	2 (FP) Type-I error	25 (TN)



- Time: ~2 min
- Say:
- For classification, we often use the confusion matrix. True positive means the actual class is positive and the model predicts positive. False negative means actual positive but predicted negative. False positive means actual negative but predicted positive. True negative means actual negative and predicted negative. From this table, we can compute accuracy, precision, recall, and F1. Accuracy is not always enough, especially with imbalanced data. For spam detection, missing spam and wrongly blocking ham may have different costs.

Improving the model

- Time: ~0.5 min
- Say:
- Now we shift from evaluating the model to improving the model. Think of this as the practical engineering part: when performance is not good enough, what can we change?

Error analysis

```
import numpy as np

w = model.linear.weight.detach().numpy().flatten()
idx2word = {i: wrd for wrd, i in vectorizer.vocabulary_.items()}
top_spam = np.argsort(-w)[:20]
top_ham = np.argsort( w)[:20]      # most negative
print("Top spam tokens :", [idx2word[i] for i in top_spam])
print("Top ham tokens  :", [idx2word[i] for i in top_ham])
```

- ❑ Top spam tokens : ['free', 'txt', 'reply', 'stop', 'claim', 'mobile', 'www', 'uk', 'service', 'won', '50', 'text', '150p', 'prize', 'win', 'new', 'com', '18', 'cash', 'landline']

- ❑ Top ham tokens : ['ok', 'da', 'later', 'll', 'lt', 'gt', 'doing', 'home', 'lol', 'said', 'work', 'lor', 'lt gt', 'really', 'cos', 'way', 'thats', 'haha', 'told', 'fine']

- Time: ~2 min
- Say:
- Error analysis means looking at what the model has learned. Here, the top spam tokens include words like free, txt, claim, prize, win, and cash. That makes sense for SMS spam. The top ham tokens include ok, later, home, lol, work, and fine. This gives us confidence that the model is using meaningful signals. But it can also reveal weakness. For example, if harmless messages contain words like account or service, the model may wrongly classify them as spam.

Error analysis

	label	pred	prob	text
5046	0	1	0.644196	We have sent JD for Customer Service cum Accounts Executive to ur mail id, For details contact us
700	0	1	0.580534	K..u also dont msg or reply to his msg..
1353	0	1	0.575595	Let Ur Heart Be Ur Compass Ur Mind Ur Map Ur Soul Ur Guide And U Will Never loose in world....gnun - Sent via WAY2SMS.COM
3328	0	1	0.526175	what number do u live at? Is it 11?
4357	0	1	0.516998	Great. So should i send you my account number.
1460	1	0	0.508713	Bought one ringtone and now getting texts costing 3 pound offering more tones etc
2352	1	0	0.505755	Download as many ringtones as u like no restrictions, 1000s 2 choose. U can even send 2 yr buddies. Txt Sir to 80082 £3
5449	1	0	0.50094	Latest News! Police station toilet stolen, cops have nothing to go on!
3302	1	0	0.482094	RCT' THNQ Adrian for U text. Rgds Vatian
3272	1	0	0.478605	You have 1 new voicemail. Please call 08719181513.

- Time: ~2.5 min
- Say:
- This table shows actual mistakes. Label is the true class, pred is the predicted class, and prob is the predicted probability. Some ham messages are predicted as spam because they contain words like mail id, contact us, account number, or message. Some spam messages are missed because they look conversational or humorous. This is why looking only at accuracy is not enough. Error examples tell us what the model misunderstands and suggest what to improve next.

What we can do to improve the model ...

- ☐ feature vector
- ☐ model architecture
- ☐ training algorithm

Only if the dataset is fixed.

If we can get additional data, then data should be the highest priority.

- Time: ~1.5 min
- Say:
- If the dataset is fixed, there are three main things we can improve: the feature vector, the model architecture, and the training algorithm. But if we can get better or more labelled data, that is usually the highest priority. In real projects, data quality often matters more than changing from one model to another. More representative examples usually improve generalisation.

Feature vector

- Increase the vector dimension
 - include more features
- Manually change some dimensions
 - feature engineering

- Time: ~1.5 min
- Say:
- Improving the feature vector means giving the model better inputs. We can increase the dimension by adding more features, such as bigrams, message length, number of digits, or presence of URLs. We can also do feature engineering: manually designing dimensions that capture useful patterns. For example, spam messages often contain phone numbers, prices, links, or urgent phrases. Good features can make even a simple model strong.

Training algorithm

- Learning rate
 - not learning at the initial phase → change to a larger learning rate
 - not converging at the later phase → change to a smaller learning rate
- Parameter initialization
 - default method is good enough
- Adam or SGD
 - Adam is more robust

- Time: ~2 min
- Say:
- The training algorithm can also be tuned. Learning rate is very important. If the model is not learning at all in the early phase, the learning rate may be too small. If the loss jumps around or does not converge later, the learning rate may be too large. Parameter initialisation usually has good defaults in modern frameworks. Adam and SGD are both optimizers, but Adam is often more robust for beginners because it adapts the learning rate for each parameter.

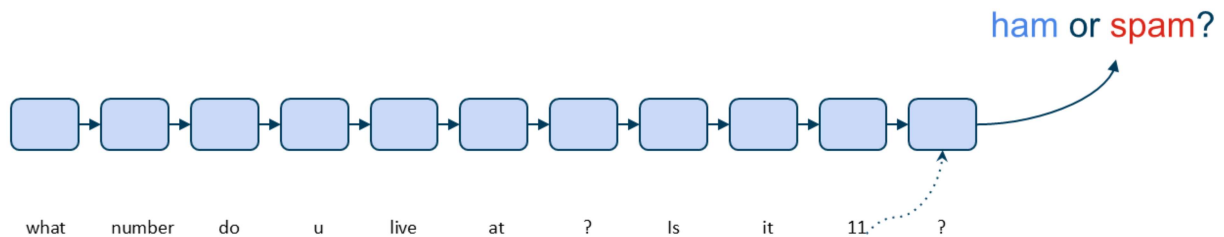
Model architecture

- Increase the number of layers
 - increase model capacity to store information
 - for smaller dataset, it is not a good idea, may lead to overfitting
- Increase the hidden state dimension
 - increase model capacity
 - same as above
- Change to another model

- Time: ~1.5 min
- Say:
- Changing the model architecture means changing the capacity of the model. More layers or larger hidden dimensions allow the model to store more patterns. But if the dataset is small, higher capacity can overfit. Overfitting means the model performs well on training data but poorly on new data. Another option is to change to a model that matches the structure of the problem better. For text, sequence order matters, so this motivates RNNs.

Why change to a different model?

- The current model is a Bag-of-Word (BoW) model
 - no sequential ordering of the text
- Recurrent Neural Network (RNN) takes a **sequential** input of **any length**



It would be great if the last state represents a summary of the entire input text.

- Time: ~2.5 min
- Say:
- The current Bag-of-Words model ignores word order. It knows which words appear, but not their sequence. For some tasks, this is enough. But for many text tasks, order matters. An RNN reads the input sequentially, one token at a time. At every step, it updates a hidden state. Ideally, the final hidden state becomes a summary of the whole message. Then we can use that summary to classify ham or spam. This is the transition from feature-count models to sequence models.

Segment 5 — The Fixed-Context Limitation

- The feedforward LM only uses the last k words
- Relevant context can be 20, 50, or 500 words back
- We need a model that reads the entire sequence → Recurrent Neural Networks

- Time: ~2 min
- Say:
- The feedforward language model only uses a fixed number of previous words, called k . But real context can be much farther back. Important information may appear 20, 50, or even 500 words earlier. We need a model that can read the whole sequence and carry information forward. This is the motivation for recurrent neural networks. They process tokens one by one and maintain a hidden state as memory.

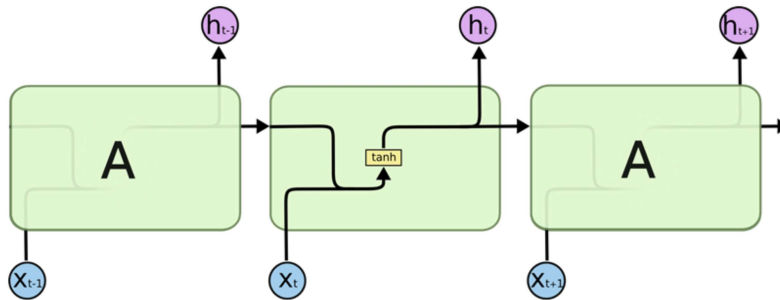
Recurrent Neural Networks

- At each time step t , the RNN:
 - 1. Takes the current input x_t (the embedding of word w_t)
 - 2. Takes the previous hidden state h_{t-1} (a summary of everything seen so far)
 - 3. Produces the new hidden state: $h_t = \tanh(Wx_t + Uh_{t-1} + b)$
- Same weight matrices W and U applied at every time step — this is what makes it "recurrent"
- Unrolled through time: a chain of cells sharing weights, h_t passed from cell t to cell $t+1$
- For classification: use the final hidden state h_t as a document representation → linear classifier



- Time: ~3 min
- Say:
- At each time step, an RNN takes two inputs: the current word embedding x_t and the previous hidden state h_{t-1} . It combines them using weight matrices and a non-linearity, usually $\tanh$, to produce the new hidden state h_t . The same weights are reused at every time step. This reuse is why it is called recurrent. If we unroll the RNN, it looks like a chain of cells. For classification, we can use the final hidden state as a representation of the full sentence and feed it into a linear classifier.

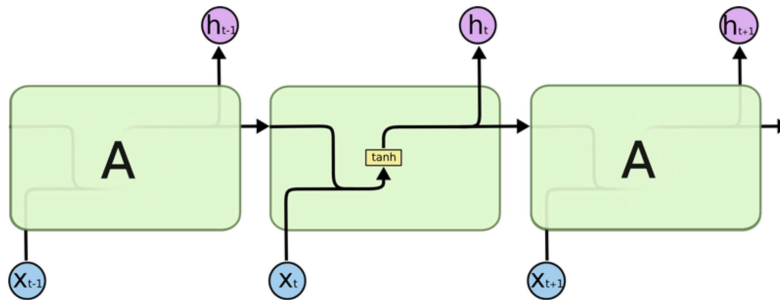
How to interpret a RNN



$$h' = \tanh(W_{ih}x + b_{ih} + W_{hh}h + b_{hh})$$

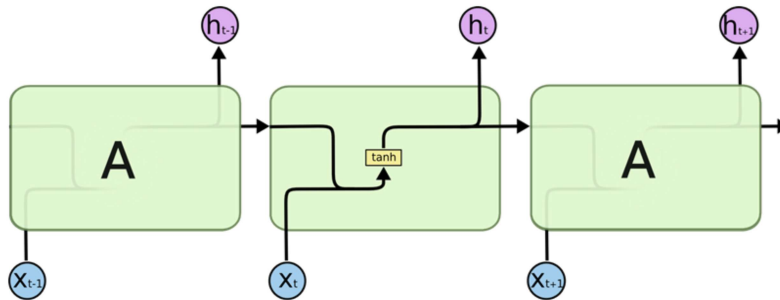
- Time: ~2 min
- Say:
- This diagram shows the unrolled RNN. Each green block is the same RNN cell reused at different time steps. The input enters from the bottom, the hidden state passes horizontally to the next cell, and outputs can be produced at each step. The formula says the new hidden state is based on the current input and the previous hidden state. The main intuition is: every step updates the model's memory of what it has read so far.

How many parameters are there in RNN?



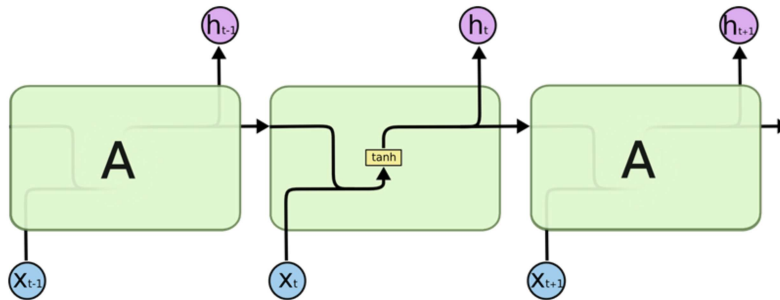
- Time: ~2 min
- Say:
- A useful question is: how many parameters does an RNN have? The number of parameters does not grow with the sequence length because the same cell is reused. The parameters come from the input-to-hidden weights, hidden-to-hidden weights, and biases. So a sentence of length 10 and a sentence of length 100 use the same parameters. This sharing is powerful because the model can apply the same reading rule at every position.

What is a suitable hidden state size?



- Time: ~1.5 min
- Say:
- Hidden state size is a design choice. A small hidden state is cheaper and less likely to overfit, but it may not store enough information. A large hidden state can capture richer patterns, but it has more parameters and may need more data. For a small undergraduate lab, we usually choose a moderate size like 64, 128, or 256. The goal is not the biggest model; the goal is a model that trains reliably and demonstrates the concept.

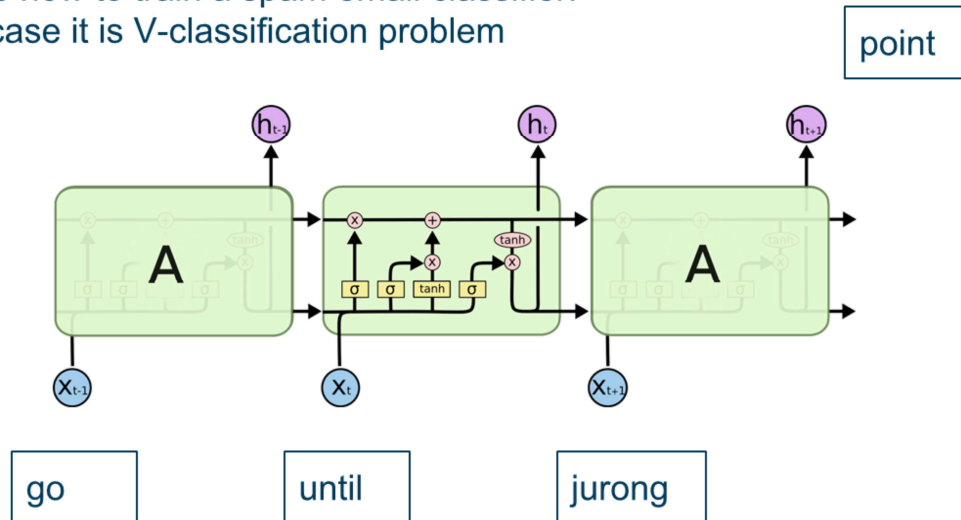
What else do we need?



- Time: ~0.75 min
- Say:
- What else do we need? We need an embedding layer to convert token IDs into vectors. We need the RNN or GRU layer to process the sequence. We need a classifier or output layer. And we need a loss function and optimizer to train the whole system end to end.

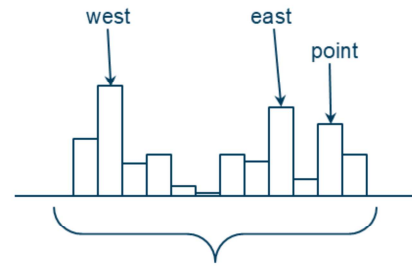
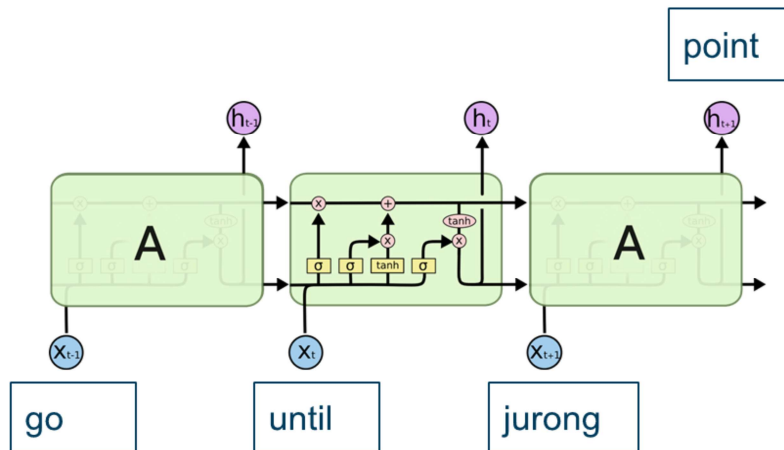
How to train a language model?

- ❑ Just like how to train a spam email classifier.
- ❑ In this case it is V-classification problem



- Time: ~2.5 min
- Say:
- Training a language model is similar to training a classifier. The difference is the label. In spam classification, the label is ham or spam. In language modelling, the label is the next word. Given 'go until jurong', the model tries to predict 'point'. If the vocabulary has V words, this becomes a V -class classification problem. Each possible next word is one class. So the output layer must produce one logit for every word in the vocabulary.

Language Modeling Objective



No of classes =
vocabulary size

- Time: ~2.5 min
- Say:
- This slide shows the language modelling objective visually. The model reads the previous words and produces a distribution over the vocabulary. The correct answer here is 'point', but the model may also assign some probability to other words like west or east. Training pushes probability toward the correct next word. The number of classes equals vocabulary size, which can be very large. This is one reason language models are computationally more expensive than small classifiers.

Recap on RNN

- ❑ Take sequential input of any length
- ❑ Apply the same weights on each step
- ❑ Can optionally produce output on each step

- Recurrent Neural Network $\neq$ Language Model
- There are more 'powerful' RNNs like LSTM, GRU

- Time: ~1.5 min
- Say:
- Let us recap RNNs. They can take sequential input of any length. They apply the same weights at each step. They can produce output at each step or only at the end. But an RNN is not automatically a language model. It becomes a language model only when we train it to predict the next token. Also, vanilla RNNs are not the only option. More powerful recurrent units include LSTM and GRU.

The Vanishing Gradient Problem

- Backpropagating through time: the gradient at step t passes through every cell from t to now
 - At each step it is multiplied by the weight matrix and the derivative of tanh
- If those multiplied quantities are < 1 (common), the gradient shrinks exponentially
 - After 50 steps, the gradient from the first word is effectively zero
- Consequence: an RNN encoding "The dog that chased the cats in the park runs fast" will likely forget the subject is "dog" by the time it reaches "runs"

- Time: ~3 min
- Say:
- The vanishing gradient problem explains why vanilla RNNs struggle with long sequences. During backpropagation through time, the gradient from later words must pass backward through many cells. At each step, it is multiplied by weights and derivatives. If these values are often less than one, the gradient shrinks exponentially. After many steps, the signal from early words becomes almost zero. So the model struggles to learn long-range dependencies, such as remembering the subject of a sentence until the verb appears much later.

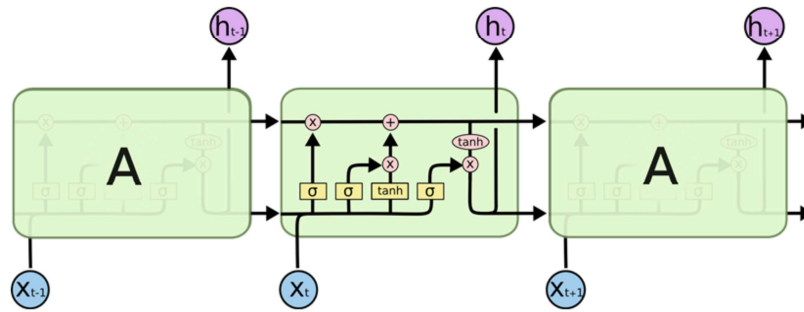
Anchor Moment — Thought Experiment

- Hidden state dimension = 128. After 100 words, how much of the first word's information survives in h_{100} ?
- Depends on the specific weights — but for a vanilla RNN, likely very little
 - The gradient signal is too weak to meaningfully update the first word's embedding
- This motivates LSTM and GRU

- Time: ~1.5 min
- Say:
- Here is a thought experiment. Suppose the hidden state has 128 dimensions and the sentence has 100 words. How much information from the first word remains at the final hidden state? For a vanilla RNN, often very little. The answer depends on the learned weights, but the training signal for early words becomes weak. This motivates gated architectures like LSTM and GRU, which create easier paths for information and gradients to flow.

LSTM vs RNN

- Layers are defined by non-linearity
- Information highway



- Time: ~1.5 min
- Say:
- LSTM was designed to solve this memory problem. The phrase 'information highway' is useful. Instead of forcing information to repeatedly pass through tanh transformations, LSTM has a cell state that can carry information forward more directly. Gates decide what to forget, what to add, and what to output. The details are more complex than a vanilla RNN, but the intuition is simple: provide a controlled path for memory.

GRU — Gated Recurrent Unit

- LSTM: forget / input / output gates; a cell state "highway" — additions instead of multiplications
- GRU: a simplified LSTM with two gates
 - $r_t = \text{sigmoid}(W_r x_t + U_r h_{t-1})$ — reset gate
 - $z_t = \text{sigmoid}(W_u x_t + U_u h_{t-1})$ — update gate
 - $\tilde{h}_t = \tanh(W x_t + U(r_t \odot h_{t-1}))$ — candidate state
 - $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$ — new state
- When $z_t \approx 0$, h_{t-1} carries forward unchanged — the gradient flows through without decay

- Time: ~3.5 min
- Say:
- GRU stands for Gated Recurrent Unit. It is similar in spirit to LSTM but simpler. It has two main gates: reset and update. The reset gate controls how much of the old state is used when creating a candidate new state. The update gate controls how much of the old state is kept versus replaced. The final hidden state is a mixture of old hidden state and candidate hidden state. If the update gate chooses to keep old information, memory can flow forward with less decay. That is why GRUs help with longer dependencies compared with vanilla RNNs.

Anchor Moment — Quick Check

- If $z_t = 0$ for all time steps: $h_t = h_{t-1}$ — the GRU always keeps the old hidden state
- If $z_t = 1$ for all time steps: $h_t = \tilde{h}_t$ — the GRU always replaces the state with the candidate
- The GRU learns to interpolate between these extremes

- Time: ~1.5 min
- Say:
- This quick check tests the update gate. If z is zero, the new hidden state equals the old hidden state, so the GRU keeps memory. If z is one, the new hidden state becomes the candidate state, so the GRU replaces memory. In practice, z is not exactly zero or one. The model learns values between them and decides how much to keep and how much to update at each step.

Segment 6 — Unidirectional vs. Bidirectional GRU

- Unidirectional: reads left-to-right; the final hidden state summarises the sequence, but has seen early words many steps ago
- Bidirectional: runs two GRUs — one left-to-right, one right-to-left — and concatenates their final hidden states
 - Captures both what came before and what comes after each position, from each direction's "final" perspective

- Time: ~2 min
- Say:
- A unidirectional GRU reads left to right. The final state has seen the whole sequence, but early words may still be far away. A bidirectional GRU uses two GRUs. One reads left to right and the other reads right to left. Their final hidden states are concatenated. This gives the classifier information from both directions. For tasks like classification or NER, bidirectional models are often stronger because they use both left and right context.

PyTorch Details for the Lab

- `nn.Embedding(vocab_size, dim)`: a lookup table — integer index in, embedding vector out
- `nn.GRU(input_size, hidden_size, bidirectional=True, batch_first=True)`:
 - returns output (all hidden states) and `h_n` (final hidden state)
 - bidirectional `h_n` has shape `(2, batch, hidden_size)`: index 0 = forward, index 1 = backward
- `torch.cat([h_n[0], h_n[1]], dim=-1)`: concatenate along the last dim → `(batch, 256)`
- `CrossEntropyLoss`: combines log-softmax and negative log-likelihood — takes raw logits, not probabilities



- Time: ~3 min
- Say:
- These are the PyTorch details students must recognise in the lab. `nn.Embedding` is a lookup table: token ID in, vector out. `nn.GRU` takes input size and hidden size, and `bidirectional=True` creates forward and backward GRUs. With `batch_first=True`, tensors are shaped as batch, sequence, features. The GRU returns output and `h_n`. output contains hidden states at every time step. `h_n` contains final hidden states. For a bidirectional GRU, `h_n` has shape 2, batch, hidden size. We concatenate `h_n[0]` and `h_n[1]`, giving a vector twice the hidden size. `CrossEntropyLoss` takes raw logits, not probabilities.

Why Train Embeddings From Scratch (Not GloVe)

- GloVe gives fixed embeddings that do not update during training
- A learned embedding layer updates its weights through backpropagation — it adapts to the specific task
- For news topic classification, task-specific embeddings can outperform generic GloVe embeddings

- Time: ~1.5 min
- Say:
- This slide explains why we train embeddings from scratch for the lab. GloVe gives pretrained embeddings, but if we use fixed GloVe vectors, they may not adapt to our specific classification task. A learned embedding layer updates through backpropagation and becomes task-specific. For this lab or TMA-style question, training from scratch also keeps the architecture aligned with the requirement and avoids overlap with questions that explicitly use pretrained embeddings.

Segment 7 — Generating Text from a Language Model

- A trained LM (feedforward, RNN, or GRU) gives $P(\text{next word} \mid \text{everything so far})$ at each step
- To generate text: sample a word, feed it back in as input, repeat
- Greedy decoding: at each step, pick the single highest-probability word, feed it back in, repeat
 - Locally greedy in the same way as tagging each word with its individually-best POS tag (Segment 2) — fails the same way

- Time: ~2.5 min
- Say:
- Now we discuss generation. A trained language model gives a probability distribution for the next word given the context so far. To generate text, we choose a next word, append it to the context, and repeat. Greedy decoding chooses the highest-probability word at every step. This is simple and fast, but it is local. Like greedy POS tagging, a locally best choice may lead to a poor full sequence later.

Decoding

What is a greedy decoding algorithm?

```
# Apply greedy decoding: get the character with the highest probability  
predicted_id = torch.argmax(predictions, dim=-1).item()
```

To generate the most likely next token based on the preceding context.

- Time: ~1.5 min
- Say:
- Greedy decoding is implemented using argmax. The model produces predictions, and torch.argmax selects the token with the largest score or probability. Then that token is used as the next input. Greedy decoding is deterministic: the same model and same prompt will usually give the same output. The advantage is speed and simplicity. The disadvantage is that it may produce repetitive or less creative text.

Beam

- What is a beam search?
- Take into consideration of the future word and keep the top candidates
- A pruned search space
- Computationally expensive for larger vocab size, beam size and depth

```
# Expand each sequence in the current beam
for score, sequence in beam:
    input_eval = torch.tensor(sequence, dtype=torch.long).unsqueeze(0)
    predictions = model(input_eval)[: , -1, :]
    log_probs = torch.log_softmax(predictions, dim=-1).squeeze(0)

    # Generate all possible next sequences
    for vocab_id, next_log_prob in enumerate(log_probs):
        new_score = score + next_log_prob.item()
        new_sequence = sequence + [vocab_id]
        all_candidates.append((new_score, new_sequence))

# Sort all candidates and select the top 'beam_width'
all_candidates.sort(key=lambda x: x[0], reverse=True)
beam = all_candidates[:beam_width]
```

- Time: ~2 min
- Say:
- Beam search keeps more than one candidate sequence. Instead of choosing only the best next token, it expands several candidates, scores them, sorts them, and keeps the top beam_width candidates. This is a pruned search space. It considers some future consequences better than greedy decoding, but it is more expensive. Cost increases with vocabulary size, beam size, and generation depth. Beam search is not exact unless the beam is extremely large, which is usually impractical.

Temperature

- ❖ For larger models that are
 - well trained
 - with larger vocabulary size
- ❖ Temperature can be a good way to increase variations in generated outputs

```
# Apply temperature to logits
predictions = predictions / temperature
# Get probabilities
probabilities = torch.softmax(predictions, dim=-1)
# Sample from the distribution
predicted_id = torch.multinomial(probabilities, num_samples=1).item()
```

- Time: ~1.5 min
- Say:
- Temperature controls randomness in generation. We divide logits by the temperature before applying softmax. Low temperature makes the distribution sharper, so the model becomes more conservative and predictable. High temperature makes the distribution flatter, so lower-probability tokens have more chance to be sampled. Temperature is useful for large well-trained models with rich vocabularies. For a weak or small model, high temperature may simply produce noisy text.

Anchor Moment — Quick Check

- If beam width $k = 1$, what does beam search reduce to?
 - Greedy decoding
- As $k \rightarrow$ vocabulary size (effectively no pruning), what does it approach?
 - Exhaustive search over all sequences — intractable in practice, which is exactly why we need a beam width

- Time: ~1.5 min
- Say:
- Here are two quick checks. If beam width is one, beam search keeps only one candidate at each step, so it becomes greedy decoding. If the beam width approaches the full vocabulary and we search deeply, it approaches exhaustive search. But exhaustive search is intractable because the number of possible sequences grows exponentially. Beam search is therefore a compromise between quality and computation.

Common Student Errors (1/2)

- Error 1: Confusing Viterbi with the forward algorithm
 - Viterbi = argmax over paths; forward algorithm = sum over all paths. Both use DP, but max vs sum.
- Error 2: Putting the emission term inside the max over previous tags
 - emit doesn't depend on prev_tag, so it's added once, after the loop, not inside it
- Error 3: Forgetting to backtrack — reporting only the final tag
 - The backpointer list is what makes the full sequence recoverable, not just the last tag
- Error 4: Using output[:, -1, :] instead of h_n
 - For a bidirectional GRU, the backward direction's "final" state is at position 0, not T
 - use h_n[0] and h_n[1]



- Time: ~2 min
- Say:
- Let us close with common student errors. First, Viterbi and forward algorithm are not the same: Viterbi uses max, forward uses sum. Second, the emission term should not be confused with the transition loop; emission depends on the current tag and word, not the previous tag. Third, do not forget backtracking; the final tag is not the full answer. Fourth, for bidirectional GRU, using output at the last time step can be wrong for the backward direction. Use h_n[0] and h_n[1] when you need the final forward and backward states.

Common Student Errors (2/2)

- Error 5: Initialising the embedding layer with GloVe weights
 - TMA Q4 requires training the embedding from scratch — GloVe init would overlap with Q2b and be unfair
- Error 6: Treating CrossEntropyLoss as requiring probabilities
 - `nn.CrossEntropyLoss` takes raw logits; applying `F.softmax()` first gives incorrect gradients
- Error 7: Assuming beam search finds the globally optimal sequence, the way Viterbi does
 - Beam search is a heuristic pruning strategy — it can discard the true best sequence if it doesn't make the top-k cut early

- Time: ~2 min
- Say:
- More common errors. For the TMA-style GRU architecture, do not initialise the embedding layer with GloVe if the requirement says train from scratch. For CrossEntropyLoss, do not apply softmax before passing logits. PyTorch's CrossEntropyLoss expects raw logits and handles log-softmax internally. Finally, beam search is not guaranteed to find the globally best sequence. It is a heuristic. If the true best sequence falls out of the top k early, beam search will never recover it.

Thank you

- Time: ~0.5 min
- Say:
- Thank you. The key takeaways are: sequence labelling predicts one label per token, Viterbi is exact dynamic programming for a small structured problem, embeddings allow neural models to generalise, RNNs read sequences, GRUs improve memory flow, and decoding controls how language models generate text. Please revisit the code carefully because most exam and lab confusion comes from tensor shapes, logits versus probabilities, and max versus sum.